Data: Exploration and Preparation

Lesson 2 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

2 October 2026 · reading time about 80 minutes

Contents

Lesson plan

Time	Segment	Material
0:00–0:08	Recap of Exercise 1, today's map	Slides 1–5
0:08–0:35	Exploratory analysis, missing values, outliers	Slides 6–21
0:35–0:55	Notebook 01, live	Slide 22
0:55–1:10	**Break**	Slide 23
1:10–1:32	Scaling and encoding	Slides 24–35
1:32–1:44	Pipelines, the model, feature engineering	Slides 36–41
1:44–2:04	Notebook 02, live	Slide 42
2:04–2:24	Leakage in preprocessing	Slides 43–52
2:24–2:46	Notebook 03, live	Slide 53
2:46–3:00	Recap, homework set, questions	Slides 54–56
	Total	**180 minutes**

---

1. From "nothing is learned before the split" to everything else

Exercise 1 is due today. Before starting anything new, it is worth naming what that exercise actually tested: not whether your model scored well, but whether you split the data before touching it and kept the scaler inside a pipeline. Every idea in today's lesson is that same rule, applied to operations you have not met yet.

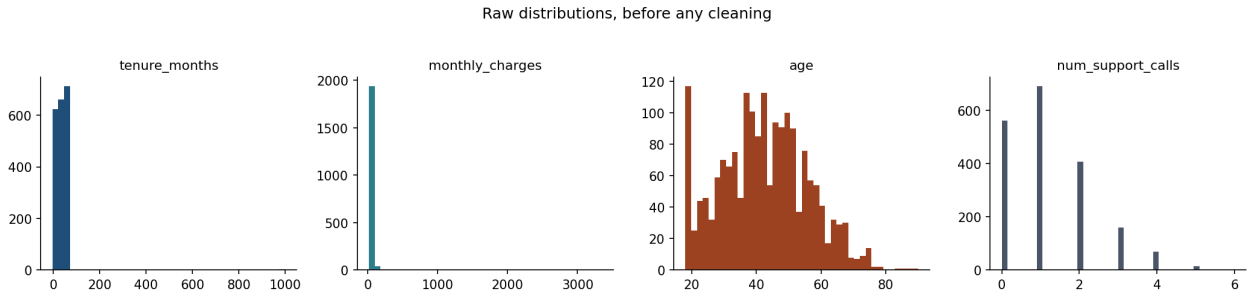
Lesson 1 established one central fact — the test set exists to give an unbiased estimate of $R(f)$, and that estimate stays unbiased only if f was chosen without ever consulting the test rows — and one central tool for enforcing it: the `Pipeline`, which fits `StandardScaler` on training data alone because it sits inside a `fit()` call restricted to the training fold. Today the tool becomes the main character. Real datasets rarely arrive as clean numeric matrices ready for `StandardScaler`. They arrive with gaps, errors, mismatched units, and categories with no numeric meaning, and every one of those has to be repaired *before* a model can be fitted at all. The question this lesson keeps asking is: which repairs *learn something from the data*, and therefore belong inside the pipeline exactly like scaling did — and which merely apply a fixed rule and are safe to do once, up front?

Get that classification right and the discipline from Lesson 1 extends for free. Get it wrong — impute a missing value, or encode a category, using statistics computed over the whole dataset — and you have leaked exactly as surely as if you had scaled before splitting, except this time nothing about the code looks unusual. That is the running example of Sections 3, 6 and 9, and it is the reason this lesson exists in the middle of the course rather than as a footnote to Lesson 1.

Throughout, we use one running dataset: 2000 telecom customers, and whether each one churned (cancelled) in the following month. It is generated by `Notebooks/churn_data.py` rather than downloaded, which buys us something no real dataset offers — we know, by construction, exactly which columns are genuinely predictive, which are pure noise, and which missing values are missing completely at random, missing at random, or would be missing not at random — the three mechanisms named in Section 3.1. Use that knowledge to check your intuition against the ground truth in Sections 3 and 9; real projects rarely afford the check.

2. Exploratory analysis

Before any transformation, look. This is not a formality; it is the step that tells you which of the remaining sections apply at all, and skipping it is how a `999`-month tenure or a category with one member ends up inside a model undetected.



*Every numeric column, before anything is done to it. What to look at is what is **missing**: `tenure_months` runs to 1000 and `monthly_charges` to 3500 with no visible bar anywhere out there, and both columns are crushed into a single spike at the left. That empty stretch of axis is the whole finding — a handful of values so extreme they have flattened the shape of everything else. Compare `age` and `num_support_calls`, drawn on their own honest range, which actually show a distribution.*

A systematic first pass answers four questions, in this order, and each answer changes what you do next.

How big is it, and what type is each column? `.shape` and `.dtypes` (or `.info()`, which gives both at once) tell you whether you are dealing with thousands of rows or millions, and whether a column pandas has read as an object is actually numeric-with-typos, a genuine category, or free text. A column of "35", "42", "unknown" is not numeric until "unknown" has been decided about — silently coercing it drops those rows without telling you.

What is the target, and how is it distributed? For classification, the class balance sets the baseline every later score is measured against, exactly as in Lesson 1. Our running example churns at 19.4%: a model that never looks at the data and always predicts "no churn" is already right 80.6% of the time, and that number is the one every later accuracy has to beat by a meaningful margin, not a decimal point.

Where are the gaps? `.isna().sum()` per column, covered in full in Section 3.

What is the shape of each numeric column, and how do columns relate to each other and to the target? Histograms surface skew; a correlation matrix surfaces which numeric columns move together and, tentatively, which look related to the target. "Tentatively" is doing real work in that sentence — a correlation is a starting hypothesis, not a conclusion, and Section 9 shows a case where it is actively misleading.

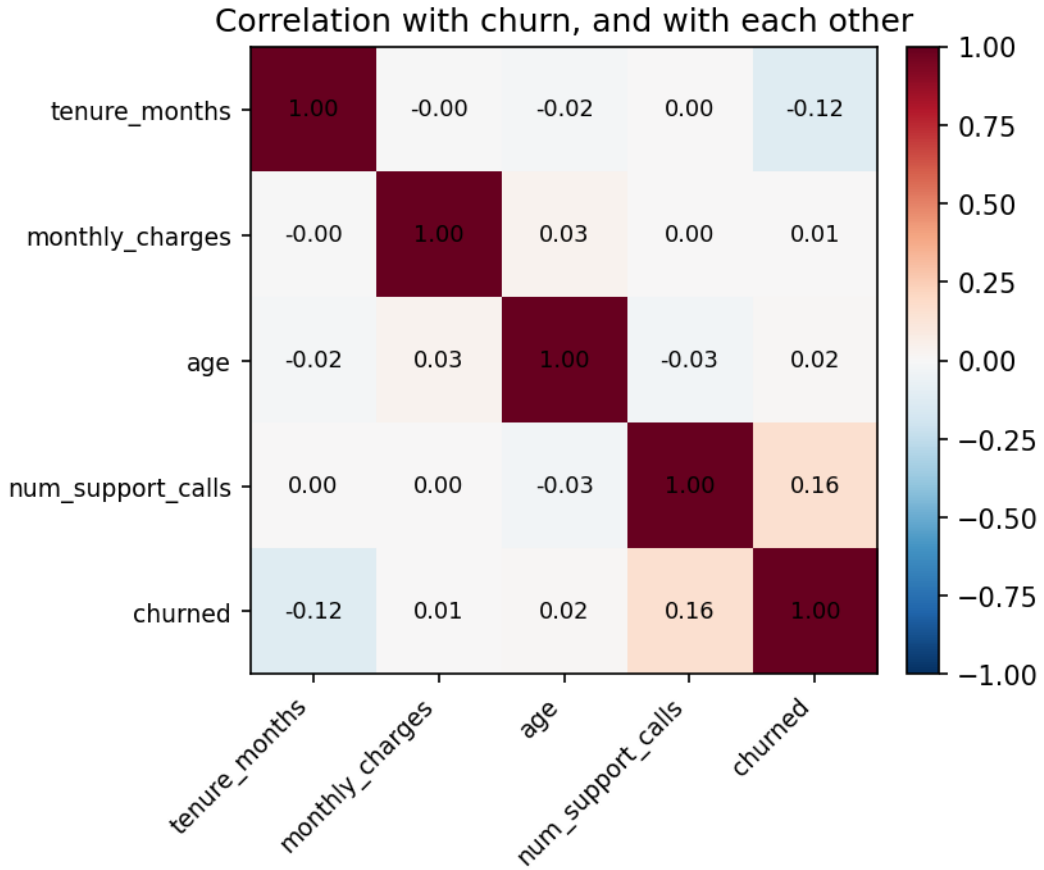
Reading the matrix. Each cell holds the **Pearson correlation** of the two columns that name it, and the intuition is a single question: *if I tell you this customer is above average on one column, how sure are you they are above average on the other?* Completely sure, and by a proportional amount, is $+1$; completely sure they are below is -1; no better off than before I told you is 0. The

measure ignores units, so it is the same number whether charges are in euros or in cents, and it is symmetric — the cell says nothing about which column would be the cause.

The last row is the one people read, and it asks what each feature might be worth. **The rest of the matrix asks a different question, about the features themselves and not about churn: how much do these two columns duplicate each other?** Two features correlated at 0.9 are very nearly one column entered twice — the model is handed two inputs but barely more than one column's worth of information, and it has two coefficients to split between them with almost nothing to say how, which is why such pairs give unstable coefficients that swing between fits. Two features near 0 are the opposite: each carries something the other cannot supply, so both earn their place.

Here every feature-against-feature cell sits within **0.035** of zero — this dataset's inputs are, by construction, unrelated to one another. That is worth naming, because it means the difficulty in what follows is *not* redundancy between columns. It is that no single column says much about the target at all.

One caution before the picture. Pearson measures a *straight-line* relationship only. A column that rises with the target and then falls again — a perfect, obvious, arch-shaped relationship — can correlate at exactly zero, and a correlation matrix will report it as nothing. A near-zero cell is a reason to plot the pair, not a proof that there is nothing there.



The four numeric columns against each other and against the target. The diagonal is 1.00 by definition — every column agrees perfectly with itself — and the matrix is symmetric about it, so

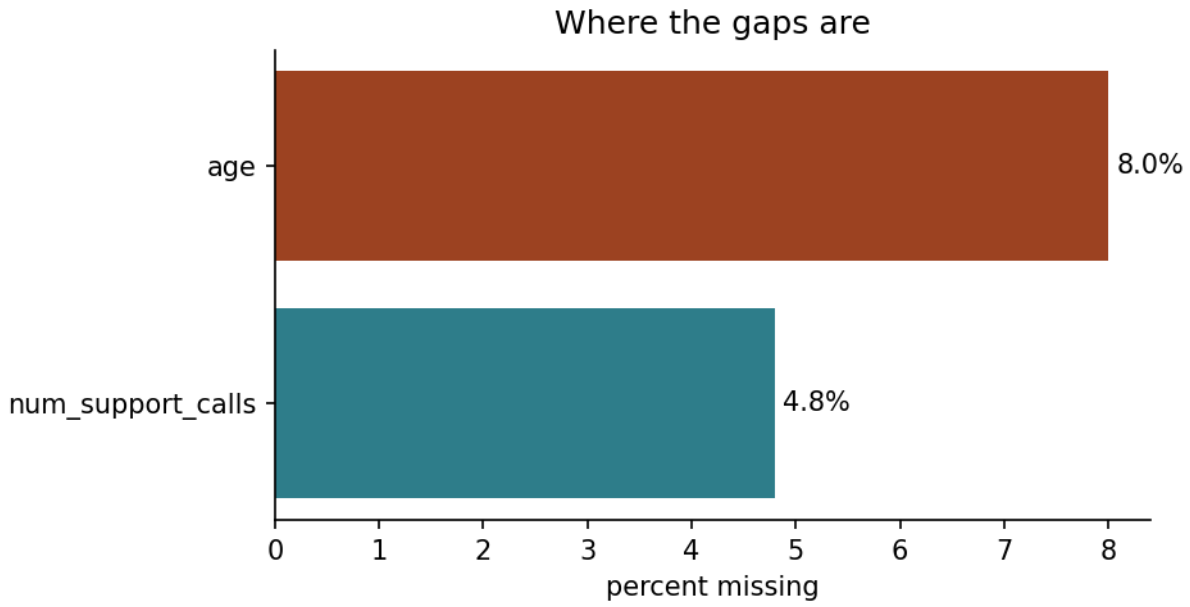
only one triangle carries information. The bottom row is the four features against churn; the block above it is the features against each other, where a large value would mean two columns saying the same thing twice. What to look at is how weak the strongest relationship is: `num_support_calls` *correlates with churn at +0.16 and* `tenure_months` *at −0.12, and everything else sits within 0.03 of zero. No single column predicts churn on its own, which is why the rest of this chapter exists — and why Section 9's leaked feature, at a correlation far above anything here, should be read as an alarm rather than a discovery.*

None of this fits a model or learns a parameter, which is exactly why it is safe to do on the *whole* dataset, including the portion that will later become the test set — looking is not fitting. The line you must not cross is using anything you see here to make a decision that changes how the model is built, before splitting. Reading that `monthly_charges` is skewed and deciding, on that basis alone, that a log transform is worth trying, is exploratory; computing the exact transform's parameters from the full dataset and applying them everywhere is not. Section 8 makes the boundary precise.

---

3. Missing values

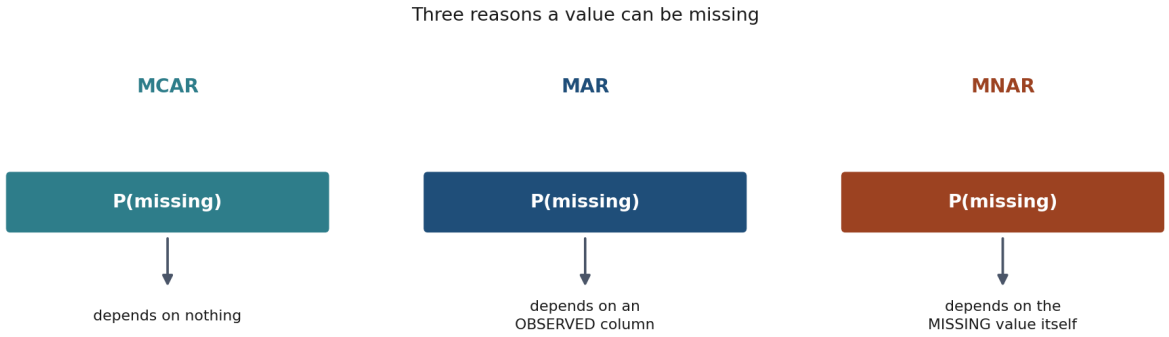
A gap in a column is not a single problem with a single fix. Before you can choose what to do about one, you have to answer a question the gap itself will not tell you: *why is this value absent?* The counting comes first — which columns, how many — but the counting is the easy half, and it is where most treatments of the subject stop. The three sections below take the question in order: what the mechanisms are, what a careless fix costs even when the mechanism is benign, and how to choose.



Where the gaps are. What a bar chart of missingness cannot show is whether the gaps are related to each other or to the target — which is exactly what decides the fix.

3.1 Three mechanisms, and why the difference matters

Not all missingness is the same, and treating it as if it were is the most common preparation mistake in practice. The standard taxonomy, due to Rubin (1976), distinguishes three mechanisms by *what the probability of being missing depends on.*



The three mechanisms side by side. The distinction is not academic: it determines which repair is valid and which quietly reweights your sample.

The notation, before the three definitions. Write $x^{(i)}$ for the whole of row i — everything the dataset holds about customer i, all n features together, the recorded values *and* the ones that are blank. Write $x_j^{(i)}$ for one cell of it, the j-th feature of that customer, and $\text{missing}_j^{(i)}$ for the yes/no fact that this particular cell is empty. One more piece: $x_{\text{obs}}^{(i)}$ is the part of the row that was actually recorded — the same customer, minus whatever is blank.

That is the whole apparatus, and the three mechanisms differ only in **what you have to condition on** to say how likely a blank is: on nothing, on the recorded part of the row, or on the missing value itself.

Missing completely at random (MCAR). The probability that feature j is missing for row i does not depend on any value, observed or not:

$$P(\text{missing}_j^{(i)} \mid x^{(i)}) = P(\text{missing}_j^{(i)})$$

Read it as: even if I showed you everything about this customer, you could not improve your guess about whether that cell is blank.

A form left blank for no systematic reason is MCAR. Under MCAR, the observed rows are a random subsample of the full data, so simple imputation (the mean, the median) introduces no bias in the estimate of the column's own mean — it does, however, still distort other things, as Section 3.2 derives.

Missing at random (MAR). The probability of being missing depends on *observed* variables, but not on the missing value itself:

$$P(\text{missing}_j^{(i)} \mid x^{(i)}) = P(\text{missing}_j^{(i)} \mid x_{\text{obs}}^{(i)})$$

Read it as: the recorded part of the row is enough — showing you the hidden value as well would tell you nothing further about whether it is hidden.

Our `num_support_calls` column is MAR: it goes missing more often for long-tenure customers, because their early call records predate the current customer relationship management (CRM) system. Conditional on `tenure_months`, which we observe, the missingness carries no further information — but ignoring tenure and treating the gaps as MCAR would be wrong, because the missing rows are *not* a random subsample of all rows; they are disproportionately long-tenure ones.

Missing not at random (MNAR). The probability of being missing depends on the missing value itself, even after conditioning on everything observed:

$$P(\text{missing}_j^{(i)} \mid x^{(i)}) \neq P(\text{missing}_j^{(i)} \mid x_{\text{obs}}^{(i)})$$

Read it as: the recorded part is *not* enough. The blank depends on the very value that is missing, so no column you hold can stand in for it.

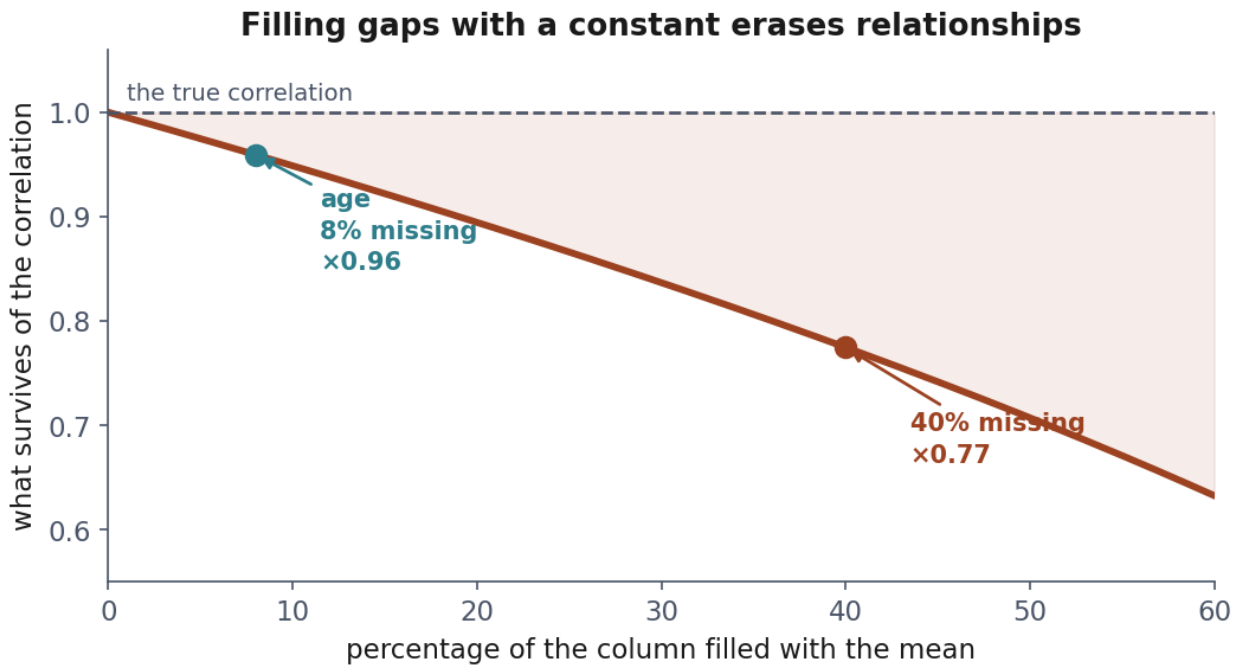
A customer who cancels because their bill was unexpectedly high, and who never finished a sign-up survey as a result, produces MNAR missingness in any field on that survey: the fact of leaving the field blank is correlated with the very value the field would have recorded. No amount of clever imputation recovers this — the information is not merely hidden, its absence *is* evidence about the value. MNAR is diagnosed from domain knowledge about how the data was collected, never from the data alone, and the honest response is usually to flag the pattern and report the limitation rather than to impute past it.

3.2 Why mean imputation is not "no information lost"

The picture first. Suppose you know the heights of a class, and for a few students you write down the class average instead of measuring them. The average height of the class is unchanged — you have not biased it. But the class now looks more uniform than it is: the invented values sit exactly in the middle, adding no spread.

Now ask whether height predicts shoe size. The students you invented contribute nothing to that relationship — their height no longer varies with anything — so the relationship looks weaker than it truly is. **You have not biased the column; you have flattened its connection to everything else.**

The algebra below says exactly how much, and the answer is a clean square root.



What filling with the mean actually costs, plotted as the fraction of a true correlation that survives. What to look at is that the curve starts at 1 and only ever falls: there is no missing fraction at which mean imputation is free. The two markers are the cases worked through below — `age` at 8% missing keeps 96% of its relationships, which is why nobody notices, and a column at 40% keeps 77%, which erases a quarter of a genuine signal without touching the column's own mean.

It is tempting to treat mean imputation as a neutral placeholder — "we did not know, so we used the average, which is the best single guess." The guess is optimal for the *column's own mean*, but it is not neutral for anything downstream. Here is the argument in full, for a column X observed on a subset and missing (MCAR, so unbiased at this first level) on the rest, and its true (unobserved) correlation with another column Y.

Let X have a fraction p of missing entries, replaced by the sample mean $\bar{X}$ of the observed part. Write the imputed column as

$$X'_i = \begin{cases} X_i & \text{observed} \\ \bar{X} & \text{missing} \end{cases}$$

A reminder, in case it has been a while. The **variance** of a column is the average squared distance of its values from their own mean: one number saying how spread out the column is. **Covariance** is the same idea for two columns at once — for each row, take how far X sits from its mean, multiply it by how far Y sits from *its* mean, and average those products over the rows. When the two are above their means together, and below together, every product is positive and the average is positive; when one is high while the other is low the products are negative; when there is no pattern the positives and negatives cancel and the average sits near zero.

That definition is enough to see the whole of what follows, before any algebra. An imputed row has X sitting *exactly* at the mean, so its deviation is zero — and zero times whatever Y does is still

zero. Such a row contributes nothing to the variance sum and nothing to the covariance sum. The derivation below is only the bookkeeping of how many rows are in that state.

One property to keep separate from correlation: covariance carries the units of both columns, so its size on its own says nothing. In our data, $\mathrm{Cov}(\texttt{tenure\_months}, \texttt{churned}) = -2.330$; measure tenure in years instead of months and the identical relationship reads -0.194. Correlation divides by both standard deviations, which cancels the units and confines the result to $[-1, 1]$: -0.123 either way. That is why Section 2's matrix reported correlations and not covariances.

The variance of X' is smaller than the true variance of X: the imputed entries contribute zero to the sum of squared deviations from the mean, while a genuine draw from X's distribution would not. Precisely, if $\mathrm{Var}(X) = \sigma_X^2$ and imputation is MCAR,

$$\mathrm{Var}(X') = (1 - p)\,\sigma_X^2$$

— the variance shrinks by exactly the missing fraction. Now consider the sample covariance between X' and a fully observed Y. Since the imputed entries all take the constant value $\bar{X}$, they contribute nothing to the covariance sum beyond what a constant contributes (zero, once centred), so

$$\mathrm{Cov}(X', Y) = (1 - p)\,\mathrm{Cov}(X, Y)$$

The correlation is the ratio of covariance to the product of standard deviations, and $\mathrm{Var}(X')$ shrank by $(1 - p)$ while $\mathrm{Cov}(X', Y)$ also shrank by $(1 - p)$ but enters the correlation as a ratio against $\sqrt{\mathrm{Var}(X')}$, not $\mathrm{Var}(X')$ itself:

$$\rho(X', Y) = \frac{(1 - p)\,\mathrm{Cov}(X, Y)}{\sqrt{(1 - p)\sigma_X^2}\,\sigma_Y} = \sqrt{1 - p}\;\rho(X, Y)$$

Mean imputation **attenuates every correlation** the imputed column has with anything else, by a factor of $\sqrt{1 - p}$, even when the missingness is perfectly MCAR and the mean itself is unbiased. With $p = 0.08$, as for `age` in our dataset, the attenuation factor is $\sqrt{0.92} \approx 0.96$ — small, and easy to ignore. With $p = 0.4$, it is $\sqrt{0.6} \approx 0.77$: a substantial fraction of a genuine relationship erased by the act of filling gaps with a constant. This is why more elaborate imputers exist — regression imputation, or the nearest-neighbour approach `KNNImputer` uses — which predict the missing value from other columns rather than replacing it with a single constant, preserving more of the covariance structure at the cost of a new place for leakage to enter, which Section 9 returns to.

Notebook 1, Section 4.1, measures this rather than asserting it: a pair with a correlation of 0.700 by construction, a fraction of one column hidden at random and filled with the mean, and what survives compared against $\sqrt{1 - p}$. The two agree to within a thousandth at every fraction tried, including the two that are ours — 4.8% for `num_support_calls` and 8% for `age`.

Where this stops holding. The derivation assumes MCAR missingness and a linear correlation measure; under MAR the attenuation is biased in a direction that depends on which subgroup is missing more, and under MNAR no correction from the observed data alone can fix it — Section 3.1 already said so, and this section is the quantitative version of the same warning.

3.3 What to do about it

- **Drop rows** only when the missing fraction is small and the mechanism is MCAR; otherwise you are silently reweighting the sample.
- **Drop the column** if it is missing too often to be useful and no proxy exists.
- **Impute** — mean or median for numeric columns (median for skewed ones, for the same reason it is preferred for outlier-robust summaries), the most frequent category or a dedicated `"missing"` level for categoricals, `KNNImputer` or a small regression model when the relationship with other columns is worth preserving.
- **Add a missingness indicator.** A binary column recording *whether* a value was missing keeps the MAR signal (the fact that long-tenure customers are more often missing a call count) available to the model even after the gap itself has been filled with a number.

How the indicator is actually used, since it is the least obvious of the four. It is not metadata and it is not a note to yourself: it is an ordinary column in the design matrix, `num_support_calls_was_missing`, holding 1 for the rows that had a gap and 0 for the rest, and the model treats it exactly as it treats any other column. A linear model gives it a coefficient, so every row that had a gap gets the same fixed shift applied to its prediction; a tree can split on it, sending the rows that were missing down one branch. Nothing has to be taught to use it — that is the whole reason for putting the fact *in the table* rather than in a comment. `SimpleImputer(add_indicator=True)` appends these columns for you, inside the pipeline, fitted on the training fold like everything else.

What it can carry is this: once the gap is filled, the model **cannot tell an invented value from a measured one**. If the fact of the gap was itself informative, imputation has just destroyed that information, and the indicator is what puts it back. Whether it *was* informative is a question about the mechanism, and Section 3.1 already answers it for our two columns:

	rows	median `tenure_months`	churn rate
`num_support_calls` missing (MAR)	96	52	**12.5%**
present	1,904	35	**19.7%**
`age` missing (MCAR)	160	—	17.5%
present	1,840	—	19.6%

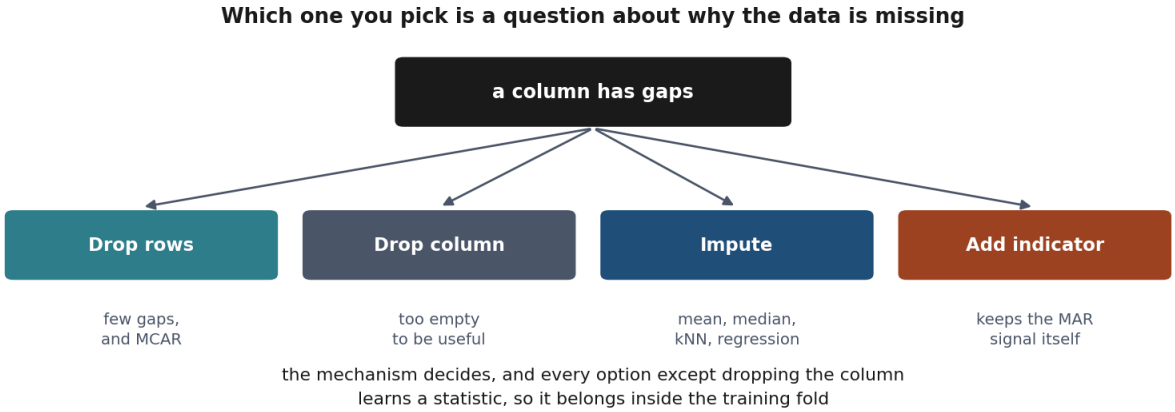
The MAR column behaves as advertised — the rows with a gap are the long-tenure ones, and they churn at two thirds the rate of everybody else — while the MCAR column's indicator separates nothing, which is what MCAR means.

And now the part that is worth more than the rule. Fit the Section 8 pipeline twice, once with `add_indicator=True` and once without, and the area under the curve goes **0.751 → 0.741**. It does not help. The reason is not that the indicator carries nothing; the table above shows that it does. It is that what it knows — *this is a long-tenure customer* — the model **already has**, in the `tenure_months` column sitting next to it. The indicator is a noisier copy of a column that is already in the table, which is Section 2's redundancy argument arriving from a different direction.

Read the drop as "no gain" rather than "harm": two hundredths of an AUC point is comfortably inside the spread a different train/test split would produce, which is the whole subject of Lesson 5. The rule that survives is sharper than "add indicators, they are free":

Add the indicator when the fact of the gap could carry something **the other columns cannot supply**. When the missingness depends on a column you already hold — the definition of MAR — expect the model to have that information already, and expect the indicator to earn nothing.

The case where it does earn its place is the one this dataset does not contain: missingness driven by something unrecorded, where the blank is the only trace left of it. That is MNAR, no imputation recovers the value, and a column saying *this was blank* is the only honest thing you can hand the model.



The decision, as a chart. Note that every branch depends on the mechanism, which is why Section 3.1 comes first.

Whichever you choose, Section 8 states the constraint that applies to every option except dropping columns outright: the statistic used to fill a gap — a mean, a median, a set of neighbours — is *learned from data*, and must be learned from the training fold only.

---

4. Outliers

What the word means. An **outlier** is a value that sits far away from the rest of the values in its column — far enough that it looks as though it might not have come from the same process as everything else. That is the whole definition, and notice what it does *not* say: it does not say the value is wrong.

That matters, because a value can be unusual for three quite different reasons, and only the first is a defect:

- **A recording error.** In our data, `tenure_months` runs from **−3 to 999** while ordinary customers sit between 0 and 72. Nobody has been a customer for minus three months, and 999 is the shape a "no value here" placeholder takes when somebody types it into a numeric column. Likewise `monthly_charges` reaches **3,344.7** against an ordinary maximum of 128.4: twenty of the 2,000 rows carry a billing error.
- **A rare but genuine value.** A customer really does pay far more than the others, because they bought everything. Throwing that row away does not clean the data; it deletes a real customer and quietly narrows what the model believes is possible.

- **A value from a different population.** A business account in a dataset of household ones. Real, correctly recorded, and still not what the model is being asked to learn about.

The uncomfortable part: the number alone cannot tell you which. A charge of 3,344.7 looks identical to the arithmetic whether it is a typo or a genuine corporate account. Deciding requires knowing how the data was recorded — which is why this section gives you rules that *flag* candidates and never rules that delete them.

Why bother at all. Because a handful of extreme values changes results out of all proportion to their number. Section 5.1 measures exactly that: the sixteen of those twenty errors that land in the training split compress every one of its other 1,484 customers into 3.4% of the min-max range. Nothing was deleted and nothing raised an error; the column was simply ruined for the model that came next.

The two rules below are detectors. What to do once something is flagged is Section 4.2's question, and it has no automatic answer.

4.1 Two rules, derived

The picture first. Both rules answer the same question — *how far from the middle is too far?* — and differ in what they use as a ruler.

The z-score measures distance in **standard deviations**, so it asks how surprising a value would be if the column were a bell curve. Tukey's rule measures distance in **quartiles**, so it asks how far a value sits beyond the bulk of the data, without assuming any shape.

That difference is why they disagree on skewed data, which is Section 4.2. The constant 1.5 is what puts the two fences in roughly the same place on data that *is* a bell curve, and the derivation below is where it comes from — along with the reason "roughly the same place" is not the same thing as "roughly the same number of points".

The z-score rule. Standardise and threshold:

$$z_i = \frac{x_i - \bar{x}}{s}, \qquad \text{flag } x_i \text{ if } |z_i| > k$$

Under a normal model this has a direct probabilistic reading: for $X \sim \mathcal{N}(\mu, \sigma^2)$, $P(|Z| > 3) \approx 0.0027$, so $k = 3$ flags roughly the most extreme 0.27% of a genuinely normal column — a principled choice, but one that inherits every assumption of normality, and Section 4.2 shows exactly how it fails.

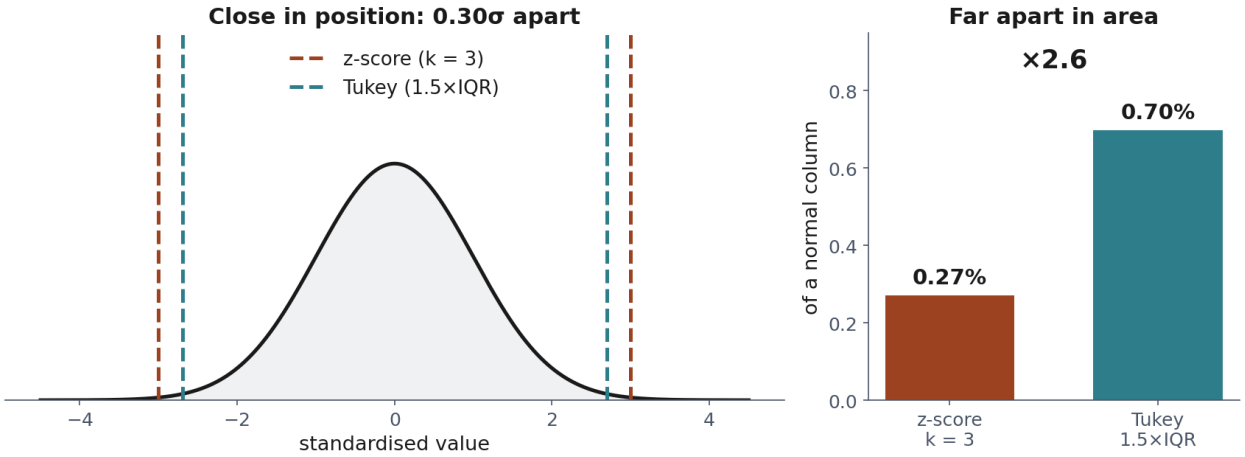
Tukey's interquartile range (IQR) rule. Let Q_1, Q_3 be the first and third quartiles and $\text{IQR} = Q_3 - Q_1$. Flag x_i outside

$$[\, Q_1 - 1.5 \cdot \text{IQR}, \quad Q_3 + 1.5 \cdot \text{IQR} \,]$$

The constant 1.5 is not arbitrary. For $X \sim \mathcal{N}(\mu, \sigma^2)$, the quartiles are $Q_1 = \mu - 0.6745\sigma$ and $Q_3 = \mu + 0.6745\sigma$ (from the standard normal quantile function Φ^{-1}, which answers "below which value does this fraction of the distribution lie": $\Phi^{-1}(0.25) \approx -0.6745$), so $\text{IQR} = 1.349\sigma$ and the upper fence sits at

$$Q_3 + 1.5 \cdot \text{IQR} = \mu + 0.6745\sigma + 1.5(1.349\sigma) = \mu + 2.698\sigma$$

so 1.5 is the constant that lands the fence at 2.698 standard deviations — just inside the z-score rule's 3, and near enough that on a bell curve the two rules are asking almost the same question.



The two fences, and why "calibrated" is not "agreed". Left: they sit 0.30 standard deviations apart, which is what makes them look interchangeable. Right: the same two rules by how much of a normal column they actually flag — 0.27% against 0.70%, a factor of 2.6. The normal tail falls away so steeply that moving a fence in by a third of a standard deviation nearly triples the area beyond it.

Almost, and it is worth seeing what "almost" costs. Compare the two rules like for like, counting both tails each time:

Rule	Fence	Fraction of a normal column flagged
z-score, $k = 3$	$\mu \pm 3\sigma$	$P(\lvert Z\rvert > 3) = 0.27\%$
Tukey, $1.5 \cdot \text{IQR}$	$\mu \pm 2.698\sigma$	$P(\lvert Z\rvert > 2.698) = 0.70\%$

The fences differ by 10% in position and by a factor of **2.6** in how much they flag. That is not a mistake in either rule; it is what the tail of a normal distribution does — it falls away so steeply that moving a fence inwards by a third of a standard deviation nearly triples the area beyond it. Put that on a column the size of this lesson's own. Take 2,000 values drawn from a perfect bell curve — no recording errors, no contamination, nothing whatever to find. The $k = 3$ rule still nominates about **5** of them ($0.27\% \times 2{,}000 = 5.4$) and Tukey's fence about **14** ($0.70\% \times 2{,}000 = 14.0$). Those points are not defects in the data: they are the price of the rule, ordinary members of the tail flagged for the crime of being in the tail. Neither rule can tell them apart from the twenty genuine billing errors, which is the whole reason a flagged value is a candidate rather than a verdict — and on clean data Tukey hands you nearly three times the pile to look through.

So the honest summary is: **the two rules are built to sit in the same neighbourhood, not to agree**, and even on data that satisfies every assumption either of them makes, Tukey's is the more willing of the two to call something an outlier. Section 4.2 is what happens when the data is not well behaved either, and it does not resolve in the direction you would expect.

4.2 Why they disagree, and why the disagreement does not settle it

The contamination argument, and it is real. Both $\bar{x}$ and s in the z-score rule are computed from the data, outliers included, so the twenty billing errors inflate s directly and widen the very threshold meant to catch them. On `monthly_charges` the damage is measurable, and it is not subtle:

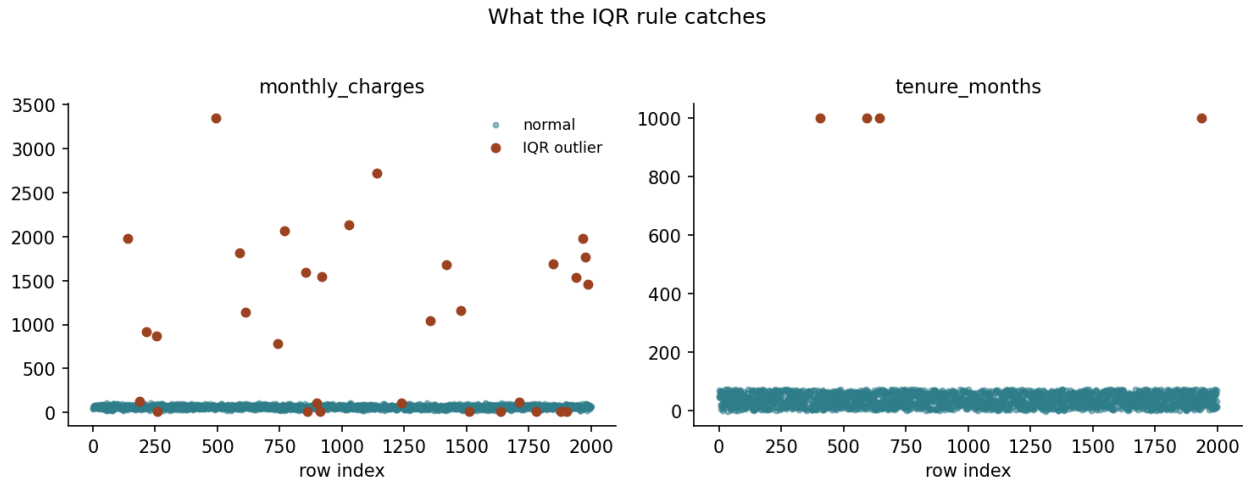
	s	where the fence $\bar{x} + 3s$ falls
the 1,980 ordinary customers alone	17.23	**115.0**
the column as it actually arrives	171.25	**593.0**

The errors inflate the standard deviation by a factor of **9.9** and push the fence out fivefold. This is the outlier-detection analogue of the leakage argument running through the whole lesson: a statistic meant to characterise "normal" is corrupted by the abnormal points it was supposed to find. Q_1 and Q_3 barely move under the same contamination, since a handful of extreme points cannot shift a quartile unless they make up more than a quarter of the column — which is the standard reason for preferring IQR on skewed, error-prone data.

Now count what each rule actually caught. That argument predicts the z-score rule will miss errors here. It does not:

	flagged	genuine errors caught	ordinary customers flagged
z-score, $k = 3$	20	**20 of 20**	0
Tukey, $1.5 \cdot \text{IQR}$	32	20 of 20	**12**

The z-score rule got every error and nothing else, with a fence that had been dragged from 115 to 593 — because the smallest billing error is **780.1**, still clear of the ruined threshold. Tukey's rule caught the same twenty and added twelve ordinary customers: eight paying 15.0 to 16.7, four paying 111.6 to 128.4, all of them real people on real tariffs, flagged because its fences sit at 17.3 and 110.0 and this column's honest spread runs wider than that.



Every point Tukey's rule flags in the two contaminated columns. What to look at is the bottom of the left panel: alongside the billing errors up at 800 and beyond, the rule has also flagged a row of perfectly ordinary customers sitting in the main band.

Sit with that, because it is the point of the section. The rule whose ingredients were destroyed returned the right answer. The robust rule made twelve mistakes. Neither outcome tells you which rule is sounder — the z-score rule was right by luck, and the luck was that these errors are an order of magnitude out rather than merely large. Shift them down towards 200 and its inflated fence would sail straight past them, exactly as the theory says.

The lesson is not "prefer IQR". It is that a detector's output does not report the detector's health. You cannot see, in a list of twenty flagged rows, that the threshold which produced it had been widened fivefold by the very rows it flagged. The way to see it is to compute s with and without the candidates, as the first table does — which is a two-line check, and one worth running before trusting either rule on a column you have not looked at.

Neither rule, however, can tell you whether a flagged point is a *mistake*. Both are purely distributional: they find values that are unusual relative to the rest of the column, and say nothing about whether a value is *possible* at all. A tenure of -3 months is impossible on its face, yet if negative tenures make up only a small fraction of a column that already spans a wide range, they are not *extreme* relative to that spread and neither rule flags them — notebook 1 shows this exactly. Catching them needs a **domain rule** (tenure cannot be negative or exceed some sane maximum), which is a different kind of check entirely: not "is this unusual" but "is this valid". A thorough pass runs both, because they catch different mistakes.

Deciding what to do once something is flagged is a fourth, separate step, and it is a domain decision, not a statistical one: cap it at the fence, remove the row, correct it if the true value is recoverable, or leave it if it is unusual but genuine. Capping is *Winsorising*, after Charles Winsor: the flagged value is replaced by the fence itself rather than deleted, so the charge of 3,344.7 becomes 110.0, the row keeps every other column it has, and the sample keeps its size — only the extremity is lost. A customer who has stayed 70 months and pays a high bill is not an error; a tenure of 999 months is.

> **Try this:** in notebook 1, lower the z-score threshold from $k = 3$ to $k = 2$ and count how many ordinary customers get swept in with the genuine errors. The threshold is a decision with a cost on both sides.

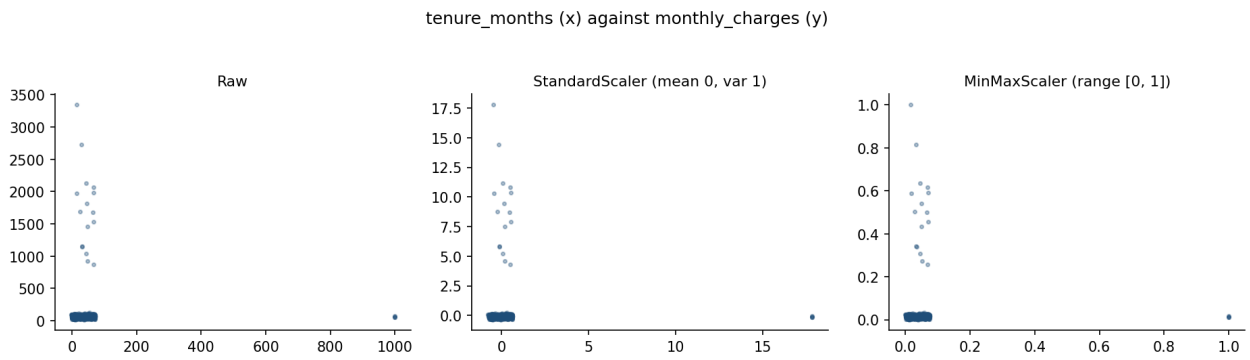
---

5. Feature scaling

5.1 Two standard transforms

$$z = \frac{x - \mu}{\sigma} \qquad\qquad x_{\text{minmax}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Standardisation centres each feature at 0 with unit variance; min-max scaling maps each feature into $[0, 1]$. Min-max scaling is more sensitive to a single extreme value, since $x_{\max}$ or $x_{\min}$ can be an outlier that compresses every ordinary value into a sliver of the range — another reason outlier handling (Section 4) precedes scaling, not the other way round.



`tenure_months` (x) against `monthly_charges` (y), raw and under each scaler. What to look at is the axis numbers, not the cloud — the three panels are the same picture, because scaling moves the ruler and not the data. On the middle panel the largest billing error sits at y 17.8 standard deviations; on the right, min-max is obliged to hand that one error the value 1.0, which leaves every ordinary customer squeezed into the bottom 3% of the axis.

"A sliver" is measurable, so measure it. In the training split, `monthly_charges` runs from 15.0 to 128.4 for the 1,484 ordinary customers and contains 16 billing errors, the largest at 3,344.7. Fit both scalers on that column as it stands and take one perfectly ordinary customer paying 80.0 a month:

	$x_{\min}$, $x_{\max}$	μ, σ	where 80.0 lands
with the 16 errors	15.0, 3344.7	81.22, 183.65	min-max **0.020** $\cdot$ z **−0.007**
without them	15.0, 128.4	63.55, 17.07	min-max **0.573** $\cdot$ z **+0.964**

Under min-max, every ordinary customer is now packed into [0.000, 0.034] — **3.4% of the range**, with the other 96.6% reserved for sixteen rows. The sliver is not a metaphor.

Standardisation is pulled in the same direction, and it is worth being precise about how much rather than waving at it: the spread it divides by grows from 17.07 to 183.65, a factor of **10.8**, while min-max's denominator grows from 113.4 to 3329.7, a factor of **29.4**. So min-max is about **three times** as badly affected here — a real difference, but not the difference in kind that "more sensitive" can suggest. Neither transform repairs an outlier. Only Section 4 does.

Scaling matters for two distinct families of method, for two different reasons. **Distance-based methods** (k-nearest neighbours, k-means, anything using a Euclidean or similar metric, all met from Lesson 6 onward) compute distances as sums of per-feature squared differences; an unscaled feature with a numerically larger range dominates that sum regardless of how informative it actually is. **Gradient-based methods** (linear and logistic regression fitted by gradient descent, and every neural network in Lessons 9 and 10) are affected for a sharper, more precise reason, derived in full below.

5.2 Why it matters for gradient descent

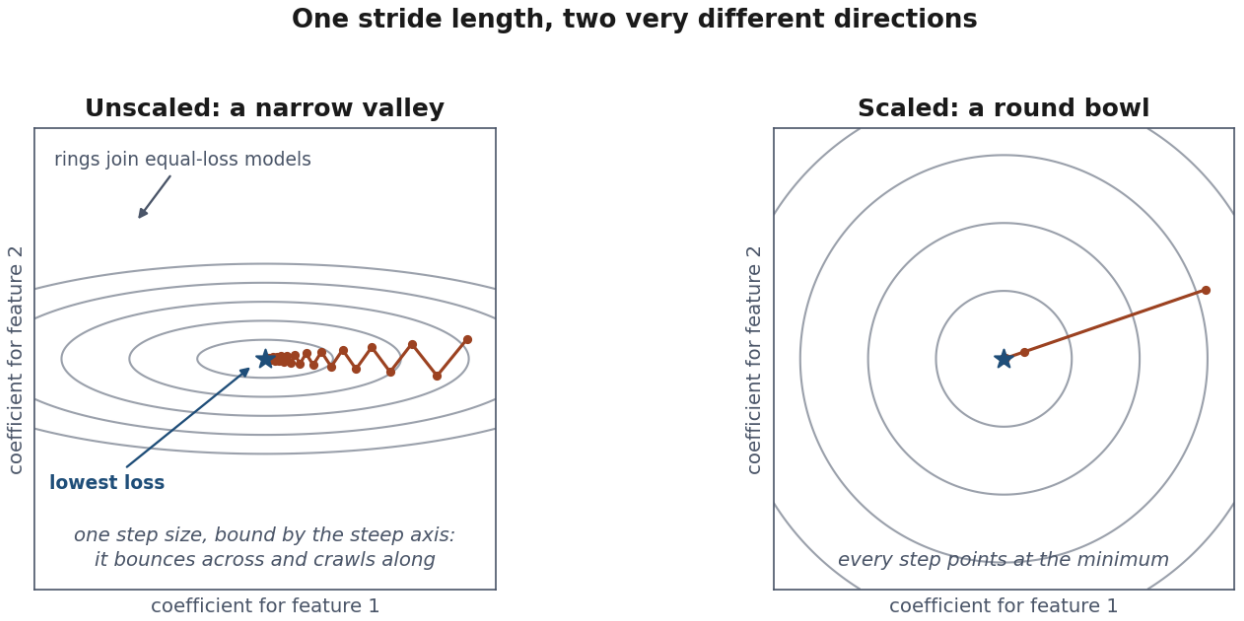
What is being minimised. Lesson 1 defined learning as choosing the f that makes the empirical risk $\hat{R}_S(f)$ — the average loss over the rows we hold — as small as possible. It left open how. For

most of this course the answer is **gradient descent**: the model is a set of numbers, one coefficient per feature plus an intercept, every choice of those numbers has a risk, and training starts from some choice and repeatedly nudges all of them downhill by the same amount — the **step size**. One step size serves the whole model, not one per feature, and that single number is the whole difficulty below.

The picture first. Think of that risk as a landscape and of each coefficient as one compass direction, so that every point of the landscape is a candidate model and its height is how badly that model fits. If one feature varies over thousands and another over units, the landscape is not a bowl but a **narrow ravine**: steep across, almost flat along.

You have one stride length for both directions. Make it long enough to make progress along the flat axis and you overshoot the steep one, bouncing from wall to wall. Make it short enough to be safe on the steep axis and you crawl along the flat one.

Scaling reshapes the ravine into a bowl, so a single stride length suits every direction.



Two features with different variances, before and after scaling. Neither axis is a column of data: each is the coefficient the model gives one feature, so every point in the square is a candidate model, the grey rings join the models that fit equally badly — contour lines of the loss, like altitude on a map — and the star is the model with the lowest loss. The rust dots are successive steps. Left: not a bowl but a ravine — steep across, almost flat along — and one stride has to serve both directions, so the path bounces off the walls while creeping towards the centre; twenty-six steps in, it has still not arrived. Right: scaled, the same problem is round, and every step points at the minimum.

What follows from that, stated rather than derived. Most models in this course are fitted by **gradient descent**: start somewhere on the landscape and take repeated steps downhill, every step the same size. Two consequences follow from the picture alone.

- **The steepest direction sets the pace for all of them.** A stride that is safe on the steep wall is the largest you may take, because a longer one overshoots and the cost goes *up*. So the feature with the largest spread decides the step size, however uninformative that feature

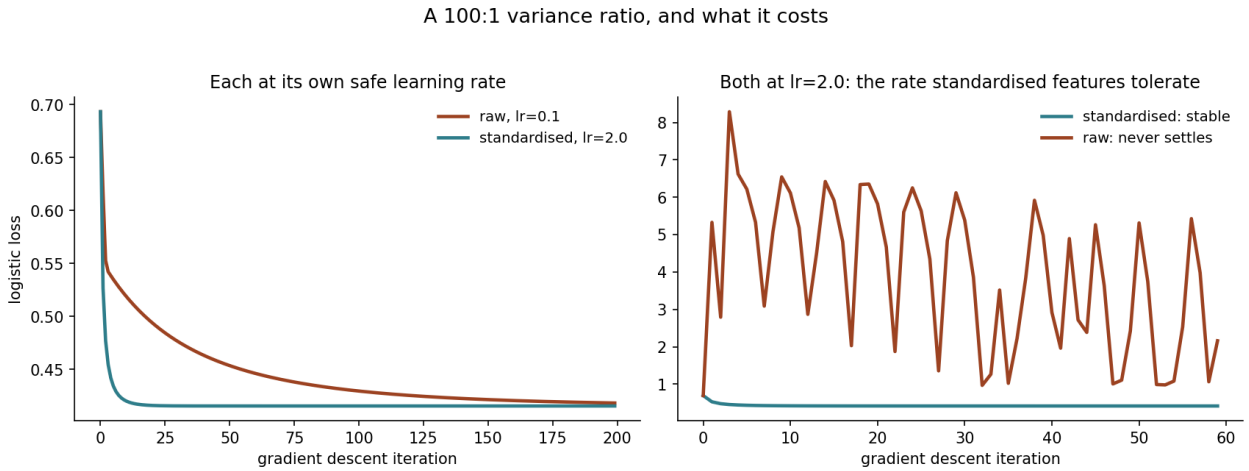
happens to be.

- **At that pace the flat direction barely moves.** How many steps you need grows with how stretched the ravine is — the ratio of the largest feature variance to the smallest, which has a name, the **condition number**.

Standardising sets every variance to 1. The ravine becomes a bowl, the ratio falls to roughly 1, and one stride suits every direction at once.

On the data. Notebook 2 builds two features with a variance ratio of 110 — standard deviations of 0.98 and 10.23 — and fits the same model twice. The scaled version is stable at a **learning rate** — the name gradient descent gives that single step size — of 2.0; the raw one needs 0.1, a rate **twenty times smaller**, and after 200 steps has reached a cost of 0.4183 against the scaled run's 0.4155.

Now give the raw features the scaled version's learning rate. It is worth being exact about what then happens, because the usual word for it is "diverges" and that is not quite right: the cost does not climb steadily away to infinity. It **never settles**. Starting from 0.6931 it swings between 0.69 and 8.29 and stays there — the mean over the first hundred steps is 3.64 and over the second hundred 3.04, so it is not even drifting anywhere. Run it ten times as long and it is still bouncing in the same band. Each step overshoots the steep axis, lands on the far wall, and overshoots back. That is worse than slow convergence rather than better: a slow run finishes eventually, and this one never does.



The same problem twice, and note the two panels are not the same experiment. Left, each version at the largest rate it tolerates — raw at 0.1, standardised at 2.0 — and after 200 steps the raw run has still not caught up. Right, both forced to the standardised version's rate of 2.0: within 60 steps the raw run is not slow, it is oscillating upwards. Scaling did not just save iterations here; it decided whether the fit finished at all.

That is the whole practical content: **scaling is not cosmetic. It changes how long training takes, and sometimes whether it finishes at all.**

Where the details live. Why the shape of that landscape is fixed by the feature covariances, and why the safe stride is exactly two over the largest variance, belongs with the derivation of gradient descent itself — Lesson 3, whose Section 4.4 computes the condition number on the real housing design matrix and finds that standardising takes it from 285 to 3.4, a factor of about 80. Nothing here depends on those steps; if you want them now, they are waiting there.

Two caveats worth carrying forward. The argument is about *gradient descent with one global step size*; optimisers that adapt a rate per parameter — Adam, met in Lesson 9 — are markedly less sensitive to this, though scaling still rarely hurts and remains the default. And the exact statement is for linear regression's squared-error cost; for logistic regression the landscape changes shape as the model moves, but features with very different spreads stretch it the same way, so the conclusion carries.

> **Try this:** in notebook 2, change the toy example's variance ratio from 100 to 9 and to 900, and note how the gap between the two learning rates that stay stable grows or shrinks. It should track the ratio directly.

---

6. Categorical encoding

A model does arithmetic. A column holding `"month-to-month"` offers nothing to do arithmetic with, so it has to become numbers — and **every way of doing that makes a claim about the categories**. The claim is the part to get right; the code is three lines either way.

Take `contract_type`. The distinct values a categorical column can take are its **levels**, so `contract_type` has three, with 2,000 rows spread across them:

	rows	churn rate
`month-to-month`	1,092	0.266
`one-year`	488	0.137
`two-year`	420	0.071

Here is what one customer on a one-year contract becomes under each encoding, and what each is asserting:

Encoding	That customer becomes	The claim being made
One-hot	`[0, 1, 0]`	the three levels are simply different; no order, no distances
Ordinal	`1`	they are ordered, **and** the gap from month-to-month to one-year equals the gap from one-year to two-year
Target	`0.137`	the level's average outcome summarises it well enough to stand in for it

The middle row is where damage usually happens, and this column shows why. The order is genuine — a one-year contract really does sit between the other two — so ordinal encoding looks safe. But the churn rate falls by 0.129 on the first step and by only 0.066 on the second: the first gap is **twice** the second, while the encoding $0, 1, 2$ tells the model they are the same size. A linear model given that column fits one coefficient for a step whose meaning changes depending on where you take it.

The three subsections take the three encodings in turn.

6.1 One-hot encoding, and the dummy variable trap, proved

The picture first. Suppose a survey asks whether you travel by car, bus or bicycle, and you record three yes/no columns. Those columns carry a hidden redundancy: knowing two of them tells you the third, because everyone answered exactly one.

Add an intercept — a column of ones — and the redundancy becomes exact: the three dummies always sum to that column. The model can now shift value between the intercept and the dummies without changing a single prediction, so there is no unique best answer, only infinitely many equally good ones.

Dropping one category fixes it by removing the spare degree of freedom. The proof below is that paragraph in matrix form.

Why all k dummies plus an intercept cannot work

contract_type	m2m	1y	2y	sum	intercept
month-to-month	**1**	0	0	**= 1**	**1**
one-year	0	**1**	0	**= 1**	**1**
two-year	0	0	**1**	**= 1**	**1**

the k dummy columns always sum to the intercept column: rank deficient by exactly one

The redundancy made visible: the dummy columns sum to the intercept column, so the design matrix loses rank by exactly one and the solution stops being unique.

A categorical column with k levels becomes k binary columns, one per level, each row having a single 1. If a model fits an intercept term — a constant column of 1s — alongside *all* k dummy columns, the design matrix becomes singular. The proof is one line: for any row i,

$$\sum_{j=1}^{k} \text{dummy}_j^{(i)} = 1 = \text{intercept}^{(i)}$$

every row's dummy columns sum to exactly 1, which is precisely the intercept column. The intercept column is therefore a linear combination of the k dummy columns (their sum), so the $k+1$ columns together have rank at most k, not $k+1$: **the design matrix is rank-deficient by exactly one.** Notebook 2 confirms this directly with `numpy.linalg.matrix_rank`. A singular or near-singular design matrix means the normal equations $(X^\top X)^{-1}X^\top y$ have no unique solution — infinitely many coefficient vectors fit the training data identically, and which one a solver returns becomes an artefact of numerical noise rather than a meaningful estimate.

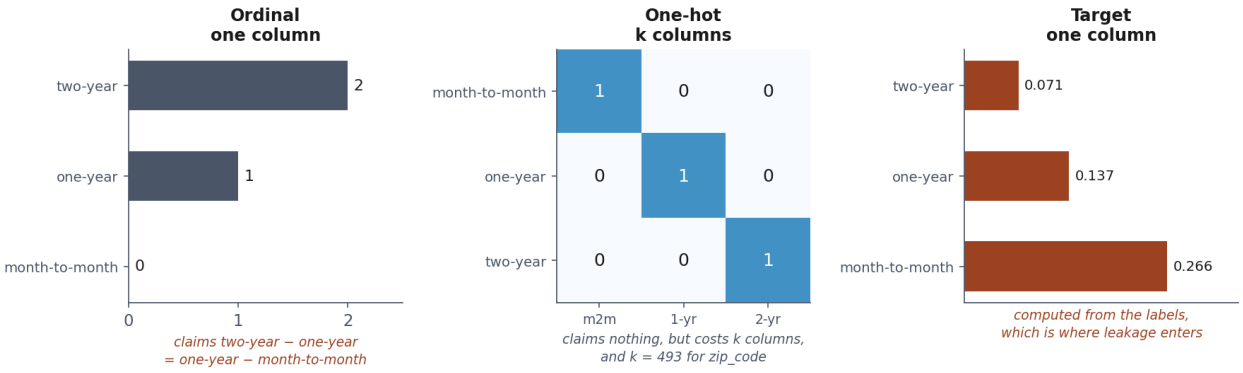
The fix is to drop one level, encoding k categories with $k-1$ columns; the dropped level becomes the *reference category*, absorbed into the intercept. This loses no information — the k-th category is

recoverable as "all other dummies are 0" — and restores full rank. `OneHotEncoder(drop="first")` implements this directly. Models that do not fit an intercept, and tree-based models met in Lesson 7, which split on one dummy at a time and do not care whether two columns carry the same information — *collinearity* — can safely use all k columns.

6.2 Ordinal and target encoding

Ordinal encoding maps categories to integers, $\{0, 1, \ldots, k-1\}$. It is appropriate only when the categories have a genuine order (`"low"`, `"medium"`, `"high"`) — imposing an arbitrary integer order on an unordered category (encoding `"North"` as 0 and `"South"` as 1) tells a model that North and South are numerically closer than North and West, which is meaningless and actively misleading for any method that uses that number arithmetically.

Target encoding replaces a category with a statistic of the target computed over the rows sharing it — typically the mean, $\bar{y}_c$ for category c. On `contract_type` that is the third column of the table above — 0.266, 0.137, 0.071 — and the appeal is immediate: three levels have become one column that already carries most of what the levels were telling us. It compresses arbitrarily many levels into a single, often highly predictive, numeric column, which is exactly why it exists for `zip_code`-shaped columns where one-hot encoding would add hundreds of *sparse* columns — columns that are zero for almost every row. It is also, computed the obvious way, the more dangerous of the two encodings in this entire lesson, and Section 9.2 derives exactly why.



`contract_type` under all three encodings. What to look at is the left panel against the right: ordinal spaces the levels 0, 1, 2 — equal steps — while the churn rates it stands in for fall 0.266, 0.137, 0.071, a first step nearly twice the second. The encoding asserts a regularity the column does not have, and a linear model has one coefficient with which to believe it.

6.3 The curse of dimensionality, briefly

The idea in one sentence. Every column you add is a question the model has to answer from the same rows you already had. Add enough of them and the rows run out: there is not enough data left per question to answer any of them reliably. That is the **curse of dimensionality** — not that high-dimensional data is hard to store, but that the evidence per dimension thins out as fast as the dimensions multiply.

One-hot encoding is the fastest way to walk into it, because a column with k levels becomes k columns and k is not yours to choose — the data picks it. Count what `zip_code` would do here. It has 493 levels, so one-hot encoding it adds 493 columns, and a linear model then has 493 coefficients to estimate. The rows do not multiply to match: the training half holds 1,500 of them, spread across

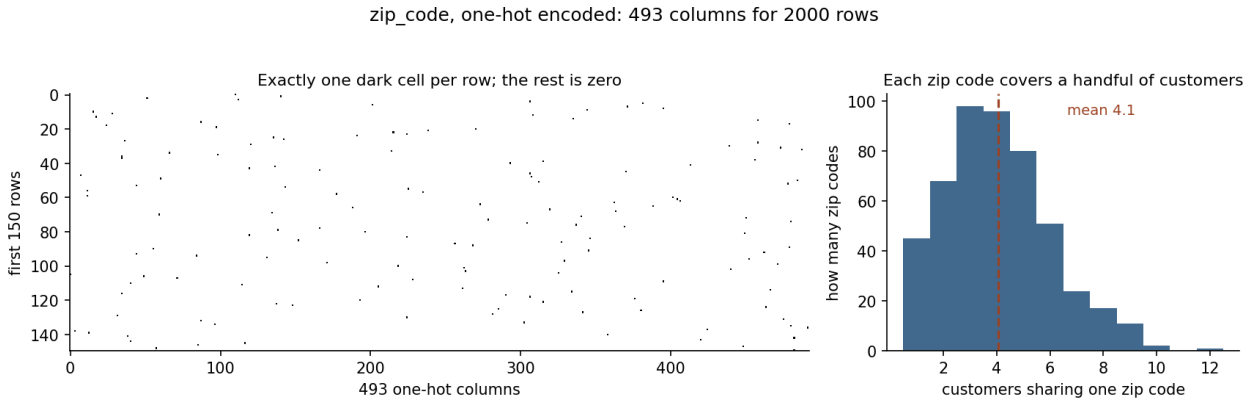
the 477 codes that appear in it. That is **3.1 rows per coefficient**, and 82 of those coefficients would be estimated from a single row each.

Two distinct things go wrong, and it is worth keeping them apart:

- **Estimation.** A coefficient fitted on three rows is mostly noise. Nothing errors, nothing warns; the model simply reports 493 numbers of which most are accidents of which customers happened to land in the training half. Lesson 5 gives this its proper name — variance — and measures it.
- **Geometry.** One-hot rows are unit vectors — a single 1 and zeros everywhere else — so any two customers with *different* codes sit at exactly the same distance from each other, $\sqrt{2}$, whatever the two codes are. Neighbouring postcodes and opposite ends of the country are equally far apart. Methods that work by distance (Lesson 6) get nothing at all from the column, because the encoding threw away the only thing distance could have used.

There is a third failure that shows up at prediction time rather than during fitting. Sixteen of the 493 codes appear in the test half and never in the training half, carrying 26 test rows between them. An encoder fitted on the training data has no column for those codes and no coefficient to apply — the model meets a category it has never seen, which is why `OneHotEncoder` takes a `handle_unknown` argument at all.

`zip_code` therefore sits squarely in the range where one-hot encoding is impractical, and target encoding — done correctly — is the standard alternative. What "correctly" means is Section 9.2, and it is the harder half of this lesson.



What one-hot encoding `zip_code` produces. Left: 493 columns, one dark cell per row and nothing else — 99.8% of the matrix is zero. Right: how many zip codes are shared by how many customers; the bar standing at 4 reaches 96, so 96 codes have exactly 4 customers each, and the average is 4.1. A column that carries four rows cannot support a coefficient estimated from those four rows, and that is the curse of dimensionality in its most concrete form.

---

7. Feature engineering

A model can only use the information present in its input columns, in the form it is presented. Linear and logistic regression, in particular, can only represent relationships that are linear in the features they are given — they cannot discover, on their own, that the *ratio* of two columns matters more than either alone, unless that ratio is computed and handed to them as a new column.

Three common constructions:

- **Ratios and differences**, when a *rate* matters more than either raw quantity — support calls per month of tenure, or a charge expressed as a multiple of what comparable customers pay. Check the units before trusting the construction: `monthly_charges / tenure_months` looks like a ratio of the same family, but `monthly_charges` is already a rate, so dividing it by months again gives euros per month *per month* — a quantity nobody would name, and a sign that the feature was reached for rather than thought through.
- **Interaction terms**, $x_j \cdot x_k$, when the *combination* of two features matters beyond their individual effects — a model with only `contract_type` and `monthly_charges` cannot represent "high charges matter more for month-to-month customers" unless that product is added explicitly. This is the one notebook 2 builds: `total_paid`, the monthly charge times the months stayed, which is what this customer has been worth so far — in euros, and interpretable as such. Note what the raw columns do to it: `tenure_months` still holds the −3 of Section 4, and multiplying by it unclipped hands the model a customer who has paid minus two hundred euros. So notebook 2 computes `monthly_charges * tenure_months.clip(lower=0)` — the clip is Section 4 arriving in Section 7, and it is the ordinary shape of feature engineering on real data, where a construction that is one line of algebra is three lines of defending it against the column you actually have.
- **Binning**, converting a continuous variable into ordered categories, which lets a linear model approximate a non-linear (e.g. non-monotonic) relationship with the target piecewise, at the cost of discarding within-bin information.

Feature engineering is a hypothesis about the domain, not a guaranteed improvement. Notebook 2's engineered `total_paid` moves the model's area under the receiver operating characteristic curve (AUC) from **0.7514 to 0.7548** — under four thousandths. Four decimals rather than three, because at three the two numbers read as a gap of 0.004 while the gap is 0.0034, and a result this small does not survive that rounding.

And now the result worth more than the feature. Build the identical column *after* applying Section 4's domain rule — `tenure_months` clipped to its legitimate 0–120 rather than merely at zero — and the gain does not shrink, it changes sign: **+0.0034 becomes −0.0075**. The improvement was never about what a customer had paid. It lived in the rows this lesson has already called broken: a tenure of 999 months multiplied by a billing error of 3,344.7 gives a `total_paid` no honest customer could have, and those rows happen to align with the target well enough to move the score. **Multiplying two contaminated columns concentrates the contamination rather than diluting it** — and the result looked like a discovery. The order in which the steps are done is part of the method, not an implementation detail.

(AUC appears repeatedly from here on, so fix it now: it is the probability that the model gives a randomly chosen churner a higher score than a randomly chosen non-churner. A coin flip scores 0.5, a perfect ranking 1.0, and unlike accuracy it does not change when the classes are imbalanced. Lesson 4 derives it properly; until then, read it as "how well does this rank the two groups apart".) Worth stating plainly: that is **not** a success, and reporting it as one would be the first step towards the reasoning Lesson 5 exists to prevent. It is a legitimate result — the hypothesis that what a customer has been worth so far predicts whether they leave was reasonable, it was tested, and the data declined it — and it is also within the range that a different train/test split could produce on its own, which is precisely why Lesson 5 will not let a single split settle a question this small.

The three constructions are worth seeing on the actual columns:

Construction	On this dataset	What it lets a linear model express
Ratio	`num_support_calls / (tenure_months + 1)`	a rate — five calls in three months is not five calls in sixty
Interaction	`monthly_charges × tenure_months`	what the customer has paid so far, which neither column states on its own; the one notebook 2 builds
Binning	`tenure_months` into 0–6, 6–24, 24+	that churn runs at 0.399, 0.314 and 0.122 across the three bands — high, lower, then far lower, a shape no single coefficient on `tenure_months` can produce

And whenever the construction involves a statistic learned from data — bin edges chosen from quantiles, an interaction scaled by a learned coefficient — Section 8's rule applies to it exactly as it applies to imputation and scaling.

---

8. The pipeline, generalised

8.1 Restating Lesson 1's argument for any preprocessing step

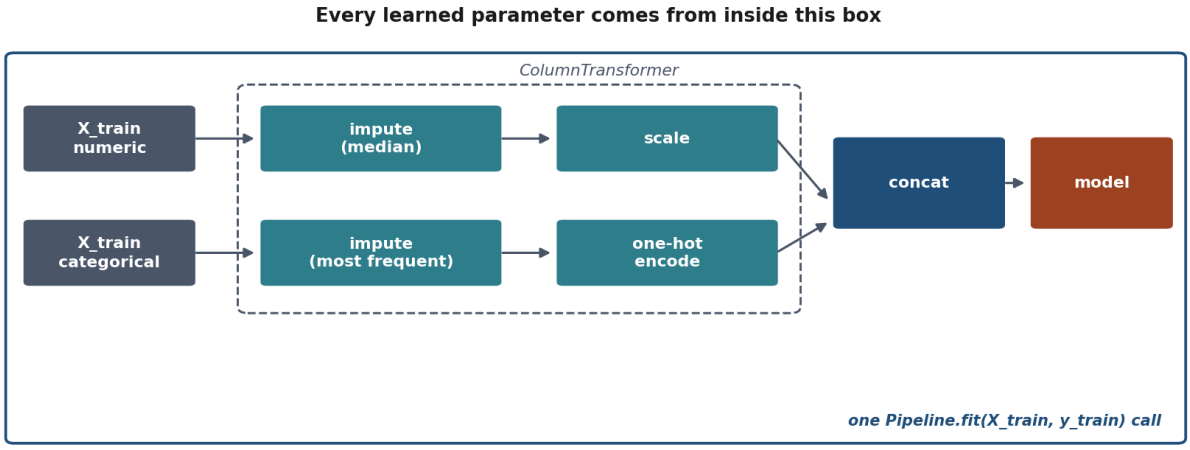
Lesson 1 showed that a test set T, independent of the fitted function f, gives an unbiased estimate of the expected risk:

$$\mathbb{E}_{T\sim\mathcal{D}^m}\left[\hat{R}_T(f)\right] = R(f)$$

Nothing in that argument mentions "the model." f is whatever function maps raw inputs to predictions, and if preprocessing is baked into that function — which it always is, in deployment, since a fitted scaler or imputer travels with the model — then f includes every preprocessing step's learned parameters: a mean, a set of neighbours, a per-category average. The independence condition applies to f as a whole, not to "the model part" of it. The moment any of those parameters is estimated using rows that later appear in T, f is no longer independent of T, the equality above fails, and $\hat{R}_T(f)$ becomes an optimistic estimate by an amount that has no simple formula — it depends on how much information leaked and through which channel. Sections 3.2 and 9.2 quantify two specific channels; there is no guarantee a third one is not lurking in a preprocessing step this lesson did not name.

8.2 `ColumnTransformer` and `Pipeline`

Different columns need different treatment — numeric columns need imputing then scaling, categorical columns need imputing then encoding. `ColumnTransformer` applies a distinct sub-pipeline to each named group of columns and concatenates the results into a single feature matrix:



The architecture that makes the rule enforceable rather than remembered: numeric and categorical branches, each fitted on training rows only, joined into one object that can be cross-validated whole.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# The two groups of columns, by name. ColumnTransformer takes lists of column
# names, so this is where you decide what counts as numeric and what does not
# - `zip_code` is a string, but it is deliberately in neither list until
# Section 9.2 has explained how to encode it.
numeric_columns = ["tenure_months", "monthly_charges", "age", "num_support_calls"]
categorical_columns = ["contract_type", "region"]

# X_train, y_train are the training half of the split, exactly as in Lesson 1.
# stratify=y makes the split keep each class in the proportion it has in the
# whole dataset - 19.4% churners here, in the training half and the test half
# alike - so a rare class cannot land unevenly by chance:
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
#                                                     random_state=7, stratify=y)

numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(drop="first", handle_unknown="ignore")),
])
preprocess = ColumnTransformer([
    ("numeric", numeric_pipe, numeric_columns),
    ("categorical", categorical_pipe, categorical_columns),
```

```
])
model = Pipeline([("preprocess", preprocess), ("classifier", LogisticRegression())])
model.fit(X_train, y_train)
```

Three details in that block are decisions rather than boilerplate.

The strings are names, not decoration. `"impute"`, `"scale"`, `"numeric"` and the rest are labels you choose. They matter once you want to reach a fitted step — `model.named_steps["preprocess"]`, and they are how a search addresses a step it wants to vary: Lesson 5 tunes `select__k` and `model__C` this way, joining the step's name to the parameter's with a double underscore. Nesting just lengthens the path, so the imputation strategy above would be `preprocess__numeric__impute__strategy`. Lesson 1's `make_pipeline` invents these names for you; here we write them, because from now on we will want to refer to them.

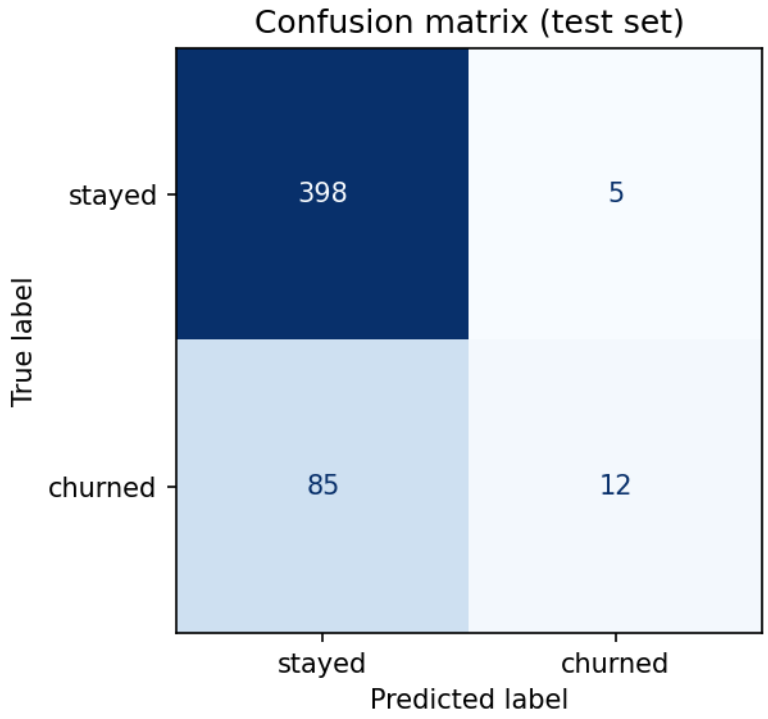
`handle_unknown="ignore"` decides what happens to a category the encoder never saw. Without it, a region absent from the training fold raises an error at prediction time; with it, that row is encoded as all zeros. Note the consequence, because it is not obvious and nothing warns you: with `drop="first"` the dropped reference category is *also* all zeros, so an unseen category and the reference category become the same input. That is usually the right trade, and it is a trade.

`median` for numeric, `most_frequent` for categorical. The median because Section 4 showed what a billing error does to a mean; most-frequent because a category has no mean. Both are statistics learned from data, which is the whole reason they are inside the pipeline rather than above it.

A single call to `model.fit(X_train, y_train)` fits every imputer, the scaler, the encoder and the classifier, **in that order, on `X_train` alone**. No step can see `X_test` during fitting, because no step is ever called on it until `model.predict(X_test)`, by which point every learned parameter is already fixed. This is what Section 8.1 demands, enforced by the object's structure rather than by the discipline of whoever writes the script — which matters, because Section 9 is about exactly the situations where discipline alone silently fails.

> **Try this:** build the same preprocessing by hand — fit a `SimpleImputer` and a `StandardScaler` as separate objects on `X_train`, then apply them to both `X_train` and `X_test` — and check the predictions match the pipeline version exactly. They should; the pipeline changes nothing about what happens, only what can accidentally go wrong.

What that pipeline actually scores. Fitted on the training fold and asked about the held-out customers, it reaches 0.820 accuracy against the 0.806 you get by predicting "no churn" every time — fourteen thousandths — and an AUC of 0.751. Those two numbers disagree about how good this model is, and the **confusion matrix** — a table counting predictions against truth, one cell per combination — shows why.



The model built on the prepared data. Read the bottom-left cell: the churners it missed. Lesson 4 gives this picture its proper treatment.

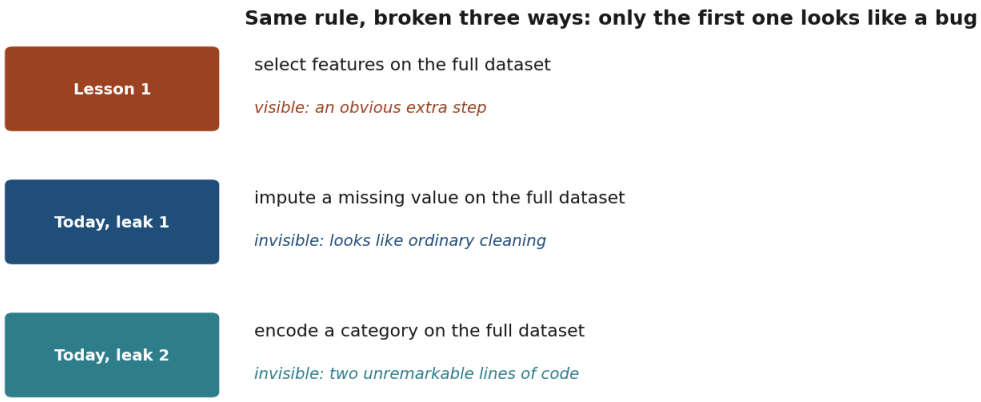
---

9. Data leakage in preprocessing

Lesson 1 gave you one rule and one example of breaking it. This section is the same rule and two more examples, and the reason it closes the lesson is that these two do not look like the first. Nobody writes `select_features(all_data)` by accident twice. Everybody, at some point, fills in a missing value or encodes a category before splitting, because neither reads as a modelling step. Both are, and both leave a mark you can measure.

9.1 The same rule, two invisible violations

Lesson 1's leakage demonstration selected features using the whole dataset before splitting — visible, once you know to look, because feature selection is obviously "using the data." Imputation and encoding are more dangerous precisely because they do not read as "training a model" at all: filling in a missing age or replacing a zip code with a number feels like data cleaning, a preparatory chore, not a learning step. It is a learning step. `KNNImputer` learns a neighbour structure; target encoding learns a per-category average. Both are statistics estimated from data, and Section 8.1's argument applies to them without modification.



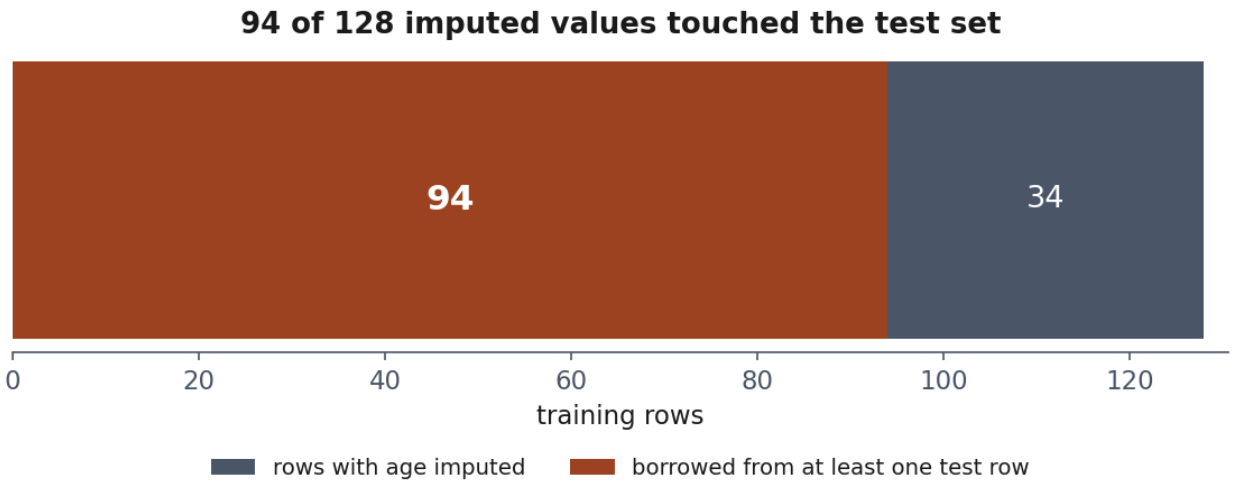
Three ways to break one rule. None of them raises an error, and all three produce a score that is better than the truth.

Imputation. A `KNNImputer` (or any imputer that uses other rows' values, rather than a fixed global statistic) fitted on the concatenation of training and test data finds neighbours anywhere in that concatenation. A training row's missing value can be filled in using a *test* row's observed value. Notebook 3 traces this directly: of 128 training rows with a missing `age`, 94 had at least one test-set row among the five donors used to fill it — measured by reproducing the imputer's own neighbour search rather than by guessing at it. Nothing raised an error.

Now the part that is easy to get wrong, and that the notebook measures rather than asserts. The effect on the final model's test AUC was 0.7553 against 0.7546 — and redrawing the split twenty times shows that gap changing sign, coming out *negative* in fourteen of the twenty. **It is noise.** The leak did not make the score better; it made it different by less than the split does.

That is not a reprieve, and the reason it happens is specific rather than general. `age` correlates with `churned` at 0.02: it is a column with nothing in it, so contaminating 94 of its values cannot move a score that barely uses it. Raising the missing fraction does not change this, because a larger share of nothing is still nothing.

So hold all three of these at once, because the combination is the lesson: the leak is **real** — 94 rows, traced individually; it is **undetectable by the metric** on this dataset; and the metric's silence is **not evidence that it did not happen**. On a real problem you have the silence and not the trace. Section 9.2 is the other case, where the same broken rule is impossible to miss.



The imputation leak, counted row by row. Of the 128 training rows whose `age` *had to be filled in, 94 borrowed a value from at least one test-set row — the red bar. Not a subtle contamination at the margin: nearly three quarters of every gap this imputer filled was filled with help from data it should never have seen.*

Encoding. This is the more dramatic of the two, and Section 9.2 derives why.

9.2 Why target encoding leaks, and why small groups make it worse

The picture first. Target encoding replaces a category with the average outcome of the rows in it. So ask what happens to a category containing exactly one row: its average *is* that row's own label, copied into a feature under a different name.

Said in one sentence, and this is the whole mechanism:

> **Your own label gets a vote in your own feature, and the fewer people share your category, the bigger your vote.**

With a category of one you are the only voter, so the feature *is* the label. With fifty, your vote is 2% and the column is a genuine statistic about the group. The size of the leak is therefore not a constant — it is one over the size of your group:

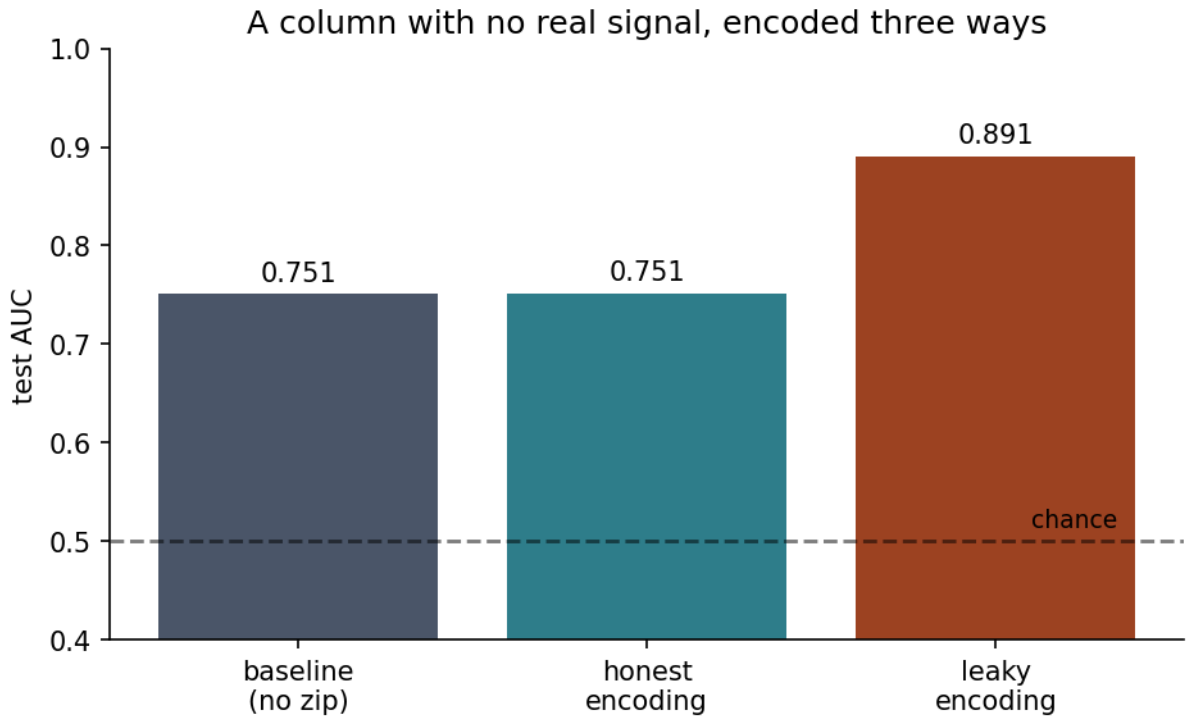
rows sharing your category	how much of your own feature is your own label
1	**100%** — the feature is the label
2	50%
5	20%
50	2%
500	0.2%

Worked, on five customers. Take a category with 5 rows, 2 of whom churned, and encode it leakily — averaging over everyone, yourself included. Every row in that category gets the feature $2/5 = \mathbf{0.40}$. Now ask what each row *should* have received, if its own label had been left out:

the row	others in the category	honest value	leaky value	pushed
a churner	1 of the other 4 churned	$1/4 = 0.25$	0.40	**+0.15 up**
a non-churner	2 of the other 4 churned	$2/4 = 0.50$	0.40	**−0.10 down**

Read the last column, because it is the point. The leak is not a vague optimism sprinkled evenly over the data: **churners are pushed up and non-churners are pushed down, every time, in every category.** The encoded column separates the two classes by construction — not because it knows anything about them, but because each row's own answer was mixed into its own question. A model then discovers a "predictor" that is a slightly blurred copy of the labels.

That is why the number in Section 9.1 is what it is. On our churn data `zip_code` has, by construction, no relationship with churn at all — Section 2's correlation check already showed that. Encoded leakily, before the split, it still drives test AUC from **0.751** for a model without it to **0.891**. Nothing was discovered. The labels were fed back in.



A column with no real signal, encoded three ways. Target encoding manufactures a predictor out of the labels themselves.

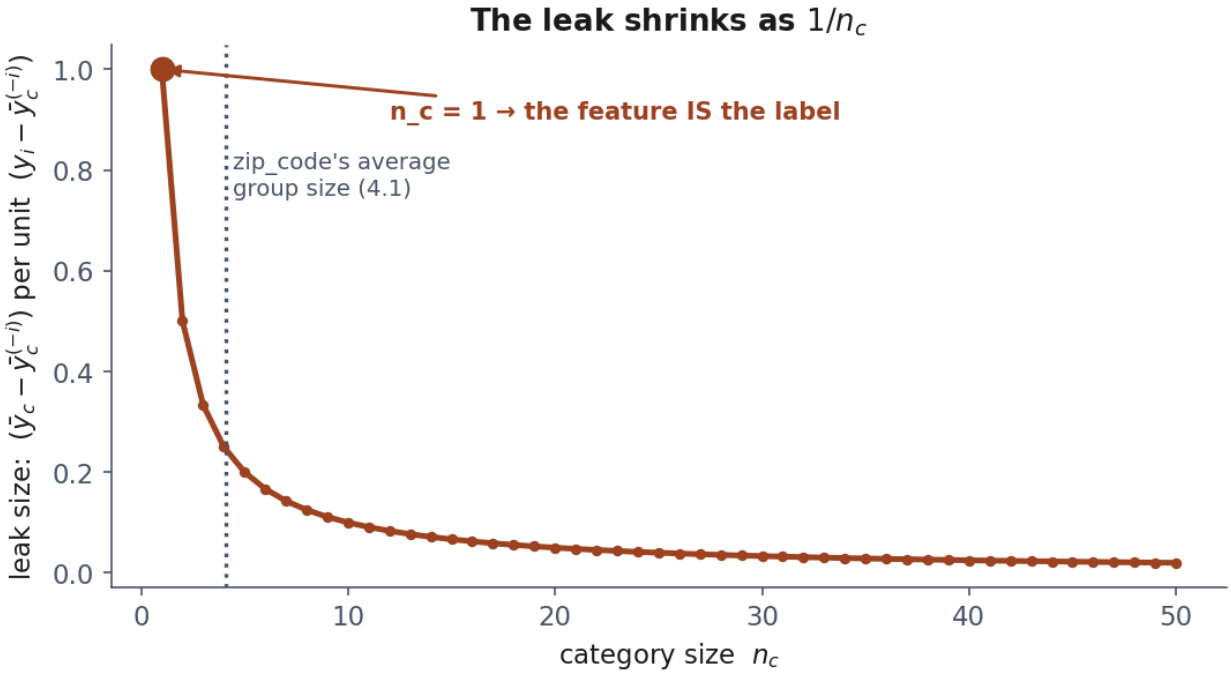
Why high-cardinality columns are the dangerous ones. More levels means fewer rows per level, and the table above says the leak grows as the groups shrink. `zip_code` averages **4.1 rows per level** (Section 6.3), so a typical customer's own label makes up roughly a quarter of that customer's own feature. Notebook 3 finds the extreme case and prints it: zip code `Z472` has exactly one customer, is encoded as `0.000`, and that customer's true `churned` value is `0`. The feature and the answer are the same number.

The fix follows from the sentence. Never let a row's own label vote on its own encoded value. `sklearn.preprocessing.TargetEncoder` does this by cross-fitting: each training row's encoded value is computed from the *other* folds of the training data, which is "leave yourself out" done a fold at a time for speed. Used inside a `Pipeline` fitted on `X_train` alone, it recovers an AUC of **0.751** — indistinguishable from not using `zip_code` at all, which is the correct answer for a column with nothing in it.

The algebra, for completeness. Nothing below is needed to use any of the above; it is the same statement in symbols. Write n_c for the number of rows in category c, $\bar{y}_c$ for the leaky average over all of them, and $\bar{y}_c^{(-i)}$ for the honest average with row i left out. The leaky average is a weighted blend of the honest one and the row's own label, and subtracting one from the other leaves

$$\bar{y}_c - \bar{y}_c^{(-i)} = \frac{y_i - \bar{y}_c^{(-i)}}{n_c}$$

which is the table: the pull is the row's own disagreement with its group, divided by the group's size. At $n_c = 1$ the right-hand side is undefined because there is no group left — and the encoding collapses to $\bar{y}_c = y_i$, the case notebook 3 prints.



The leak is worst where the groups are smallest, falling as $1/n_c$ — which is also where a high-cardinality column keeps most of its categories.

One honest caveat. This describes the realistic bug: an encoding computed once over training and test data together. An encoding computed on the training fold alone is still slightly optimistic *for the training rows*, by the same arithmetic — but those values are only ever fitted to, never scored on, so that particular bias does not reach the test number. It matters more for models flexible enough to exploit it, such as Lesson 7's boosted trees, than for a single linear coefficient.

9.3 The rule, restated

Everything that learns from data — a mean, a median, a set of neighbours, a per-category target average, a set of bin edges — belongs inside the fold it is fitted on. This is not a longer list of special cases to remember; it is one test, applied to every preprocessing step in this lesson: *does fitting this step compute anything from the rows it is given?* If yes, it goes inside the pipeline. Lesson 5 returns to leakage a third time, in the context of cross-validation — splitting the training data repeatedly rather than once — where the same test applies once more, this time to the settings chosen *around* a model rather than fitted inside it: its **hyperparameters**.

---

10. Before the next lesson

1. Work through the three notebooks in order — 01 for exploration, missing values and outliers; 02 for scaling, encoding and the pipeline; 03 for the two leaks.
2. Take the quiz in `Quizzes/`.
3. **Complete the homework** in `Exercises/02_data_exploration_and_preparation.md`, discussed at the start of Lesson 3, **Friday 9 October 2026**.

Notation used in this lesson

Symbol	Meaning
X, Y	a feature and the target treated as random variables (Sections 3–5); X is **also** the $m \times n$ design matrix in Sections 6.1 and 8.1
$x^{(i)}$	the i-th example: the whole of row i, all n features together (Sections 3.1 and 6.1)
$x_j^{(i)}$, $x_{\text{obs}}^{(i)}$	one cell of that row, and the part of the row that was actually recorded (Section 3.1)
$\text{missing}_j^{(i)}$	whether that cell is blank (Section 3.1)
x_i	in Section 4 only, and **not** a row: the i-th value of the single column being screened for outliers
m, n	number of examples, number of features
μ, σ	a feature's mean and standard deviation
$\bar{x}$	a sample mean
ρ	the correlation between two variables
p	the fraction of a column's values that are missing (Section 3.2)
$P(\cdot)$	a probability
k	the number of categories in a column (Section 6); **and**, in Section 4 only, the z-score cut-off, $k = 3$

Further reading

Resource	Type	Why read it
scikit-learn: `ColumnTransformer`	Official docs	The canonical reference for mixed numeric/categorical preprocessing
scikit-learn: `TargetEncoder` user guide	Official docs	How cross-fitting is implemented, and its parameters
Rubin, *Inference and Missing Data* (1976)	Paper	The original MCAR/MAR/MNAR taxonomy, still the standard reference
Kohavi & Provost, *Glossary of Terms* (1998), entry on "leakage"	Paper	An early, still-cited naming of the leakage problem this lesson extends
Micci-Barreca, *A Preprocessing Scheme for High-Cardinality Categorical Attributes* (2001)	Paper	The original target-encoding paper, including the shrinkage idea behind cross-fitting
Géron, *Hands-On Machine Learning*, ch. 2	Book	A complementary treatment of the full preparation pipeline, with a different worked example